Write-up: Model Documentation

1. Who are you?

- Name: Shiqi Li
- Hometown: Hangzhou, Zhejiang, China

Multimedia algorithm engineer with experience in video enhancement, algorithm engineering, and large language model / multimodal technologies.

2. Motivation

Children's speech differs greatly from adult speech — more variable pronunciation, frequent disfluencies, and inconsistent vocabulary. This competition was a good opportunity to explore adapting modern speech foundation models to this domain. I hope the solution can be useful to the research community.

3. High-Level Approach

The solution is a full-parameter fine-tuning of Qwen3-ASR-1.7B on clean, non-augmented data.

Data:

- Competition training set: ~86k utterances (children's speech with orthographic transcriptions)
- TalkBank children's speech corpus: ~255k utterances
- No data augmentation of any kind — hence "pure" in the experiment naming
- Combined, shuffled (seed=42), and split 97/3 into train (328,830 samples) / eval (10,231 samples)

Model:

- Base model: Qwen3-ASR-1.7B (Qwen3-ASR architecture with a Whisper-style audio encoder + Qwen3 1.7B text decoder)
- Full-parameter fine-tuning (no LoRA, no adapter, no frozen layers)
- Training format: `language English<asr_text>{transcription}`

Training:

- 2 epochs (stopped at best checkpoint, step 16,000 out of 18,270 total)
- Effective batch size: 36 (micro-batch 3 × gradient accumulation 12)
- Learning rate: 1e-5 with linear decay, 2% warmup

- Precision: bf16 with FlashAttention 2
- Gradient checkpointing enabled for memory efficiency on a single RTX 3090

Inference:

- vLLM backend for fast batched inference
- Greedy decoding (temperature ≈ 0, do_sample=False)
- Batch size: 64, sorted by audio duration (longest first) for efficient padding

Why this approach:

- Qwen3-ASR is a strong pre-trained ASR model with multilingual capability. Fine-tuning on domain-specific children's speech data was the most straightforward way to adapt it.
- TalkBank provides a large corpus of children's speech that closely matches the competition domain, effectively tripling the training data.
- Full-parameter fine-tuning was chosen over LoRA/adapter methods because the 1.7B model fits within a single 24GB GPU with gradient checkpointing, and full fine-tuning generally yields better results when compute allows.
- No data augmentation was used because the base model already has strong robustness from pre-training, and clean data fine-tuning proved sufficient.

Results:

- Private leaderboard: Clean WER 0.2115 / Noisy WER 0.4919
- Best checkpoint eval_loss: 0.1936 (step 16,000, epoch ~1.75)

4. Visualizations

Eval loss progression during training:

Step	**Epoch**	**Eval Loss**
2,000	0.22	0.2257
4,000	0.44	0.2077
6,000	0.66	0.2015
8,000	0.88	0.1982
10,000	1.09	0.1961
12,000	1.31	0.1953

Step	Epoch	Eval Loss
14,000	1.53	0.1941
16,000	1.75	0.1936

The eval loss decreased steadily and began to plateau around step 14,000–16,000 (epoch 1.5–1.75), indicating the model was approaching convergence. Step 16,000 was selected as the best checkpoint.

5. Three Most Impactful Code Segments

5.1 Audio pre-caching to .npy (run_train.py)

Python

```python
def precache_audio(jsonl_paths, sr=16000, num_workers=8):
    """Pre-decode audio files to .npy for fast training-time loading."""
    audio_paths = set()
    for jsonl_path in jsonl_paths:
        with open(jsonl_path, "r") as f:
            for line in f:
                if line.strip():
                    audio_paths.add(json.loads(line)["audio"])
    to_cache = [p for p in audio_paths
                if not os.path.exists(p.rsplit(".", 1)[0] + ".npy")]
    with ProcessPoolExecutor(max_workers=num_workers) as pool:
        futures = {pool.submit(_cache_one_audio, (p, sr)): p for p in to_cache}
        for fut in as_completed(futures):
            fut.result()
```

This pre-decodes all FLAC/WAV audio files to raw numpy arrays before training starts. During training, loading a .npy file is ~10x faster than decoding FLAC on-the-fly, which significantly reduces data loading bottleneck on a single-GPU setup with 328k training samples.

5.2 Data collator with prefix masking (run_train.py)

Python

```python
@dataclass
class DataCollatorForQwen3ASR:
    processor: Any
    sampling_rate: int = 16000
    max_length: int = 0  # 0 = no truncation
```

```
def __call__(self, features: List[Dict[str, Any]]) -> Dict[str, torch.Tensor]:
    audio_paths = [f["audio"] for f in features]
    prefix_texts = [f["prefix_text"] for f in features]
    targets = [f["target"] for f in features]

    eos = self.processor.tokenizer.eos_token or ""
    full_texts = [pfx + tgt + eos for pfx, tgt in zip(prefix_texts, targets)]
    audios = [load_audio(p, sr=self.sampling_rate) for p in audio_paths]

    truncation = self.max_length > 0
    trunc_kwargs = {"max_length": self.max_length} if truncation else {}

    full_inputs = self.processor(
        text=full_texts, audio=audios,
        return_tensors="pt", padding=True, truncation=truncation,
        **trunc_kwargs,
    )
    prefix_inputs = self.processor(
        text=prefix_texts, audio=audios,
        return_tensors="pt", padding=True, truncation=truncation,
        **trunc_kwargs,
    )

    prefix_lens = prefix_inputs["attention_mask"].sum(dim=1).tolist()
    labels = full_inputs["input_ids"].clone()
    for i, pl in enumerate(prefix_lens):
        labels[i, :pl] = -100

    pad_id = self.processor.tokenizer.pad_token_id
    if pad_id is not None:
        labels[labels == pad_id] = -100

    full_inputs["labels"] = labels
    return full_inputs
```

This collator constructs the full input (system prompt + audio tokens + target transcription) and masks the prefix portion in the labels so the model only learns to predict the transcription text. This is critical for instruction-tuned ASR — without prefix masking, the model would waste capacity learning to predict the prompt tokens.

5.3 Inference with vLLM and duration-sorted batching (run_inference.py)

```python
items.sort(key=lambda x: x["audio_duration_sec"], reverse=True)
model = Qwen3ASRModel.LLM(
    model=MODEL_PATH,
    gpu_memory_utilization=0.85,
    max_inference_batch_size=64,
    max_new_tokens=256,
)
for batch in batched(items, BATCH_SIZE):
    results = model.transcribe(audio=audio_paths, language="English")
```

Using vLLM instead of the default transformers generate() provides significant speedup through continuous batching and PagedAttention. Sorting utterances by duration (longest first) minimizes padding waste within each batch. Together, this reduces inference time from ~1 hour to ~15 minutes for ~10k utterances.

6. Machine Specs & Time

- CPU: Intel i9-10850K (10C/20T, 3.6–5.2 GHz)
- GPU: NVIDIA RTX 3090 24 GB
- Memory: 64 GB DDR4
- OS: Ubuntu 22.04, Linux x86_64
- Train duration: ~20 hours (16,000 steps to best checkpoint, single GPU; manually stopped after eval plateau)
- Inference duration: ~15 minutes for ~10k utterances (vLLM, batch_size=64)

7. Caveats & Known Issues

- The model requires `qwen-asr[vllm]==0.0.6` and `transformers==4.57.6`. The Qwen3-ASR architecture uses custom model classes (`Qwen3ASRForConditionalGeneration`) that must be registered — the `qwen_asr` package handles this automatically.
- FlashAttention 2 (`flash-attn>=2.8.0`) is strongly recommended for both training and inference. Without it, memory usage increases significantly and speed drops.
- The model was fine-tuned on English children's speech only. While the base Qwen3-ASR supports 30 languages, this fine-tuned version is optimized for English and may have degraded performance on other languages.
- Audio inputs are expected at 16kHz sample rate. The preprocessor handles resampling, but providing 16kHz audio avoids unnecessary computation.

- For inference, vLLM backend is recommended over transformers for speed. Set `gpu_memory_utilization=0.85` to leave headroom for audio processing.

8. Tools for Data Preparation / EDA

No. All data preparation is included in the submitted code:

- `prepare_train_data.py` — converts competition transcripts
- `prepare_talkbank.py` — converts TalkBank transcripts
- `prepare_pure.py` — merges, shuffles, and splits the data

9. Performance Evaluation Beyond Competition Metric

- Monitored eval_loss on a held-out 3% split (10,231 samples) during training to select the best checkpoint
- Evaluated WER on a manually held-out test set of 2,000 samples using `eval_wer.py` with the Whisper English text normalizer (`EnglishTextNormalizer`) for consistent text normalization
- Also computed CER (Character Error Rate) alongside WER for finer-grained analysis

10. Things Tried That Didn't Make Final

- Qwen3-ASR-0.6B: Lower capacity, worse WER than 1.7B.
- Large-scale augmented data (820k samples, WORLD vocoder pitch shift + noise + time stretch): Leaderboard WER (Clean 0.2133 / Noisy 0.5039) was worse than pure data, suggesting augmentation artifacts hurt more than they helped.
- Noise adaptation from pure checkpoint (RIR room simulation + babble noise, 10% augmented): Marginal eval_loss improvement (0.1936 → 0.1920), no meaningful leaderboard gain.
- Various augmentation ratios (10% / 18% / 20%) and strategies (SpecAugment, concat utterances, two-stage training): All degraded clean WER compared to the pure-data baseline.

The simplest approach — pure data, no augmentation — consistently won on the leaderboard.

11. Future Improvements

- Better noise domain matching: The main gap is Noisy WER (0.4919). All augmentation attempts (RIR, babble, SpecAugment, multi-step chains) improved local noisy test but not the competition leaderboard, indicating a domain mismatch. Collecting or

synthesizing noise that better matches the actual test conditions would be the highest-impact direction.
- Multi-GPU training for higher capacity.

12. Simplifications for Faster Inference

- Use vLLM with larger batch sizes on GPUs with more memory (e.g., A100 80GB) — inference would scale nearly linearly